Knowledge Distillation and Deployment of a Command-Parsing Language Model on a Local Cloud

Name: [[YOUR NAME]] **Roll no.:** [[ROLL_NUMBER]] **Course:** CS3081 Cloud Computing **Repository:** [[GITHUB_URL]] (tag `final`)

Convention. *Measured* = produced by a command whose output is stored in `results/` (file named in brackets). *Assumed/expected* = reasoning that I did not measure; marked "(assumption)". Hardware: [[CPU model, cores, RAM, OS]], single-node k3d cluster.

Task 1. Object storage and parallel teacher labelling

Architecture. `k3d` cluster from `cluster/k3d.yaml` (1 server node; host ports 8080 → Traefik, 8333 → SeaweedFS S3). SeaweedFS (single Pod, PVC) runs in namespace `data` with buckets `datasets` and `models`. The teacher, `phi3.5` served by Ollama (temperature 0, seed 42), runs on the laptop; workers reach it at `host.k3d.internal:11434`. An Indexed Kubernetes Job in namespace `pipeline` starts two Pods in parallel; `SHARD_INDEX` is the Job completion index and `SHARD_COUNT=2`. Credentials come from Kubernetes Secrets generated at deploy time (not in Git).

Teacher prompt (`prompts/teacher_prompt.txt`): states the four keys, the eight intents, the three slot rules, and gives 8 hand-written examples (one per intent, none from the validation set); the command is placed last so Ollama can reuse the cached static prefix. I also pass Ollama's `format: json` so replies are syntactically valid JSON (a decoding constraint, not an edit of the reply); this means rejection reason (a) is expected to be rare and the teacher's JSON validity in Task 2 is partly by construction.

Sharding (≤ ½ page). Worker k of N reads `commands_pool.jsonl` from the bucket and keeps line n (counted from 0) iff $n \bmod N = k$. The N residue classes partition {0..2999}, so each command is processed by exactly one worker: no gaps, no repeats, no coordination between workers. Round-robin by line number balances load when commands differ in difficulty. Each worker writes its own object (`datasets/teacher/shard-k.jsonl`), so there are no write conflicts; it re-uploads every 25 records and resumes from its object if the Pod restarts (labelling takes hours). `clean.py` verifies that the union of shards covers every pool id exactly once before cleaning. Both workers share one Ollama instance, so total throughput is bounded by the laptop CPU; parallelism here demonstrates the pattern rather than speeding up. Measured: shard sizes [[1500/1500]], labelling wall time [[hh:mm]] [`results/teacher_workers.log`, `results/pipeline_pods.txt`].

Cleaning counts [`results/clean_counts.json`]. Rules are applied in order (a)→(d) and each record is counted once under the first rule that fires; duplicates (same text, smallest id kept) are counted among the records that passed (a)–(d).

input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	duplicates	kept
[[3000]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]

Check: input = a+b+c+d+duplicates+kept → [[sum]] = [[3000]]. `train.sha256` = [[sha256 of train.jsonl]] (re-running `clean.py` reproduces it).

Task 2. Training the student (sequence-level distillation)

Setup. `EleutherAI/pythia-70m` (GPT-NeoX, 70 M parameters, Pile-trained, no instruction tuning, Apache-2.0) is fully fine-tuned on the first 1,500 records of `train.jsonl`, text `Command: <cmd>\nJSON: <teacher JSON><eos>`; the loss is computed only on the JSON part. `training/config.yaml` records seed 42, lr 3e-4, batch 8, 6 epochs, max length 96, and library versions; v2 (`config_v2.yaml`) uses seed 7 and 8 epochs. `make train` reads only the config; `make train SUBSET=50` runs the first 50 records. Training time on CPU: v1 [[mm:ss]], v2 [[mm:ss]]. Per-epoch loss: [[epoch losses v1]] [`training/loss.csv`]. Artefacts: `models/v1/`, `models/v2/` in the `models` bucket; release assets `model-v1.tar.gz`, `model-v2.tar.gz`; `models/v1/model.sha256` = [[sha]] equals `sha256sum` of the Release asset (checked).

Suitability of Pythia-70M (assumption, to be compared with the measurements). The task is narrow and the output format is rigid, so a small model can memorise the format and learn intent cues. The weak point is slot extraction: copying the shortest span exactly needs a copy mechanism and decent tokenisation of numbers/names, which 70 M parameters pre-trained only for next-token prediction on the Pile may handle poorly. A 2,048-token context and BPE vocabulary are more than enough for commands of 5–15 words.

Results on `validation.jsonl` (100 commands; `tools/score.py`, percent) [`results/*_validation.json`, raw outputs `results/*_predictions.jsonl`]:

metric	teacher phi3.5	Pythia-70M untrained	student v1	student v2
json_validity	[[ ]]	[[ ]]	[[ ]]	[[ ]]
intent_accuracy	[[ ]]	[[ ]]	[[ ]]	[[ ]]
date_accuracy	[[ ]]	[[ ]]	[[ ]]	[[ ]]
time_accuracy	[[ ]]	[[ ]]	[[ ]]	[[ ]]
place_accuracy	[[ ]]	[[ ]]	[[ ]]	[[ ]]
full_frame	[[ ]]	[[ ]]	[[ ]]	[[ ]]

What changed. [[Write 4–6 sentences from YOUR numbers: the untrained model has ~0 validity because it was never shown the format; training raised validity to X and full-frame to Y; compare to the teacher: the student is at Z % of the teacher's full-frame; which slot is worst (date/time/place) and an example error from the predictions file; v1 vs v2 difference and whether it is larger than seed noise. Do not claim more than the numbers show.]]

Sequence-level vs logit-based distillation (≤ 1 page). *(a) Sequence-level* (Kim & Rush, 2016): the teacher decodes an output sequence for every input (here greedy at temperature 0), and the student is trained with ordinary

cross-entropy on that sequence as if it were the label; the student only ever sees the teacher's *text*. *(b) Logit-based* (e.g. TRL's distillation trainer): at every output position the student is trained to match the teacher's whole next-token probability distribution over the vocabulary, typically with a KL-divergence loss on the logits, which transmits "dark knowledge" about the near-miss alternatives, not only the argmax. *(c) Why not here:* the two models do not share a token space: Phi-3.5-mini uses a ~32 k SentencePiece vocabulary and Pythia a ~50 k GPT-NeoX BPE vocabulary, so position *t* and token *i* of one distribution have no counterpart in the other; moreover the teacher is served through Ollama's text-generation API, which returns text rather than the full per-position vocabulary distribution, and storing full distributions for 1,500 examples would also be large. Mapping between vocabularies would need an alignment method that is outside the scope of this project. *(d) Limitation of what I did:* the student's ceiling is the *teacher*, not the ground truth: wherever phi3.5 mis-copies a slot or picks a wrong intent (and passes the cleaning filters), the student is trained to repeat that error; additionally with only 1,500 examples and one decode per command, the student sees no information about teacher uncertainty. [[Add one concrete example of a teacher error that survived cleaning, from your data.]]

Task 3. The inference service in a container

Design. FastAPI/uvicorn (`service/app.py`), port 8000. A background thread downloads `MODEL_URI` (e.g. `s3://models/v1/`) from `S3_ENDPOINT` and loads the model; `/healthz` is 200 whenever the process is alive; `/readyz` is 503 until loading finishes. `POST /parse` builds the training-time prompt `Command: <text>\nJSON:`, decodes greedily (`MAX_NEW_TOKENS`), validates the text against `schema.json` (exactly four keys, intent in the eight, slots string-or-null) and returns exactly the object with headers `X-Model-Version` (`MODEL_VERSION`) and `X-Served-By` (Pod name via the downward API). Invalid output → HTTP 500 `{"error":"invalid_model_output"}`, never broken JSON with 200. `FAULT_INJECT=1` replaces the output with `not json` before validation. A lock serialises generation (1 CPU per Pod). Credentials come only from the `s3-credentials` Secret (`S3_ACCESS_KEY`, `S3_SECRET_KEY`).

Container. `python:3.11.9-slim`, CPU-only wheel `torch==2.5.1+cpu`, all dependencies pinned (`service/requirements.txt`), no weights (they are in object storage, `.dockerignore` excludes `models/`). Built with one command: `docker build -f service/Dockerfile -t cmdparser:1.0 .`. Measured image size: [[ ]] MB (limit 2 GB) [`results/image_size.txt`].

Evidence. FAULT_INJECT=1 → [[HTTP 500 + body]] [`results/fault_inject.txt`]. 50 sequential requests with `--cpus 1 --memory 1g`: [[50/50]] returned 200, OOMKilled=[[false]], **peak memory [[ ]] MiB** of 1024 MiB [`results/mem_test.log`, `docker stats` samples every 0.5 s]. [[If the laptop could not run this test, state what was changed and attach evidence.]]

Task 4. Kubernetes deployment, scaling and reliability

Manifests (`k8s/30-app.yaml`, namespace `app`): Deployment (2 replicas; requests 500 m CPU/768 Mi, limits 1 CPU/1 Gi; rolling update `maxSurge 1, maxUnavailable 0`; `preStop sleep 5`), Service `cmdparser` (80→8000), Traefik Ingress `/parse`, HPA (min 1, max 2, 70 % CPU of the request; scale-down stabilisation shortened to 60 s for the demo). Liveness `/healthz`; readiness `/readyz`, so a Pod with an unloaded model is not in the Service endpoints.

Check	Result (measured)	Evidence file
`/parse` through Ingress	[[HTTP 200 + headers]]	`results/rolling_before.txt`
Service has no ready endpoints while model loads	[[`kubectl get endpoints` shows `<none>` for X s]]	`results/endpoints_while_loading.txt`
HPA scale 1→2 under load, back to 1	[[time to scale up / down, CPU % at trigger]]	`results/hpa_events.log`, `hpa_describe.txt`
Rolling update v1→v2 under load	[[requests, failures, failure %]] (need ≤ 0.5 %); versions served [[v1 n, v2 m]]	`results/rolling_*`
After `rollout undo`	`X-Model-Version` = [[v1]]	`results/rolling_after_undo.txt`
Pod deleted under load	replicas restored after [[ ]] s; failures [[ ]] % (≤ 2 %)	`results/recovery_*`
Cold start, 3 runs (all Pods deleted)	[[t1, t2, t3]] s, **median [[ ]] s**; logs show "downloaded s3://models/v1/…"	`results/coldstart*.{log,txt}`
Load balancing (2 Ready Pods, standard command)	Pod A [[n (%)]], Pod B [[n (%)]] (each ≥ 30 %)	`results/lb_headers.json`

Cold start is Pod `Ready` transition time minus creation time (1 s resolution), slowest Pod of each run.

Limitations (stated explicitly). The cluster is a *single node* on one laptop: if the machine or the Docker daemon stops, everything stops, so this does not demonstrate fault tolerance across machines (node failure, zone failure, rescheduling to another node). Both replicas and SeaweedFS share the same CPU cores, so a second replica adds availability for a Pod crash/rollout but little extra capacity, and the HPA's "scale-out" mostly shifts CPU from one Pod to another [[comment on your observed throughput with 1 vs 2 replicas]]. SeaweedFS is a single Pod on a local PVC (single point of failure). The metrics pipeline lags by ~15–30 s, so HPA reacts slowly.

Task 5. Subnetting and network access control

10.20.0.0/16 allocated largest-first (full working in `network/subnets.md`):

Subnet	Hosts	Network/prefix	Usable range	Broadcast	Usable
app	1,000	10.20.0.0/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254

data	100	10.20.5.0/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

NACL (`network/nacl.json`, public subnet). Inbound: 90 deny `198.51.100.0/24` (all protocols); 100 allow TCP 80; 110 allow TCP 443; 120 allow TCP 22 from 10.20.5.128/27; 130 allow TCP 1024–65535 from 10.20.0.0/22 (replies from the app subnet). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). Everything else falls through to the implicit deny. Measured: `nacl_sim.py … --check` → 6/6 (100 %) [`results/nacl_check.txt`], plus extra hand-made packets in `tests/test_network.py`.

Why separate reply rules; why order matters (≤ ½ page). A NACL is stateless: it does not remember that an inbound request was allowed, so the response is an independent packet that must itself match an allow rule in the other direction. A client's side of a connection uses an ephemeral port (1024–65535), so replies need an outbound rule for ephemeral ports to clients (110) and an inbound ephemeral rule for the answers coming back from the app subnet (130); without them, requests would arrive but responses would be dropped. Rules are checked from the lowest number and the first match decides. Addresses in 198.51.100.0/24 are also inside "0.0.0.0/0", so if the deny had a number *higher* than the allow rules for 80/443 (e.g. 200 vs 100/110) a packet from 198.51.100.9 to port 443 would match the allow first and the deny would never be evaluated. Numbering the deny 90, lower than every allow, makes it win.

Task 6. Reproducibility, testing and CI/CD

One-command deployment. `make up` (fresh clone): venv, `k3d cluster create`, namespaces, generated Secrets, SeaweedFS, model download from the Release with SHA-256 verification, upload to `models/v1`, `v2`, `docker build`, `k3d image import`, apply manifests, wait for rollout. `make down` runs `k3d cluster delete`. Measured: fresh-clone `make up` took [[mm:ss]] and ended with [[2/2 Ready]] [`results/make_up.log`]; `make down` [[removed cluster]] [`results/make_down.log`].

CI (`.github/workflows/ci.yml`), stages in order: *lint* (ruff) → *unit tests* (29 tests: cleaning rules and counts identity, sharding, service/validation/FAULT_INJECT, subnets, NACL, cost, packaging) and *secret scan* (gitleaks 8.18.4, must report nothing) → *evaluate-model* (downloads `model-v1.tar.gz` from the Release, verifies the checksum, runs the model on `validation.jsonl`, `score.py`) → *quality-gate* (`score.py --min-validity 95`). Green run: [[URL]]. **Failed run with the untrained Pythia-70M (gate fails): [[URL]]** (JSON validity [[ ]] % < 95). Secret scan result: [[no leaks found]].

`AI_USE.md` lists the AI tools used, for what, and how the output was checked.

Task 7. Performance, cost and review

Standard load test (4 users, 2 min, through the Ingress, 2 replicas) [`results/loadtest_stats.csv`, `results/loadtest_headers.json`, `results/baseline_top.log`]:

p50	p95	throughput	requests / failures	error rate	cold start (median, Task 4)
[[ ]] ms	[[ ]] ms	[[ ]] req/s	[[ ]] / [[ ]]	[[ ]] %	[[ ]] s

`kubectl top pods -n app` during the test: Pod A [[cpu m / mem Mi]], Pod B [[cpu m / mem Mi]].

Cost (`rate_card.json`: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; billed on limits; `scripts/cost.py`): hourly cost = 2 replicas × (1 × 3.50 + 1 × 0.45) = **7.90 INR/h** (given). Requests per hour = [[rps]] × 3600 = [[ ]]. Cost per 1,000 requests = 7.90 / [[ ]] × 1000 = **[[ ]] INR**. This is an estimate; no real charge is incurred.

SLO. "99 % of `/parse` requests return HTTP 200 within 30 s", window = one standard load-test run. Total requests [[N]] → error budget = 1 % × N = [[ ]] requests; failures = [[ ]]; budget used = [[ ]] %.

Review (each evidence cell is a measured value):

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	[[kept X of 3000; teacher labelling Y h]]	[[teacher errors survive the filters; single-Pod SeaweedFS]]	[[replicated storage; filter against a gold dev set]]
Model quality	[[full_frame v1 X % vs teacher Y %]]	[[…]]	[[more data, logit/feature distillation, constrained decoding]]
Latency and throughput	[[p50 / p95 / req/s]]	[[1 CPU, greedy HF generate, no batching]]	[[ONNX/int8, KV-cache reuse, batching]]
Resource utilisation	[[CPU m / mem Mi per Pod; peak MiB in Task 3]]	[[requests/limits vs actual use]]	[[tune requests; smaller dtype]]
Reliability and recovery	[[recovery X s, Y % failed; rollout Z % failed]]	[[single node]]	[[multi-node, PodDisruptionBudget, topology spread]]
Security and cost	[[cost per 1000 = X INR; secrets only in K8s Secrets]]	[[S3 admin key shared by all Pods; plain HTTP]]	[[per-service least-privilege keys, TLS, NetworkPolicies]]

References

Kim & Rush (2016), *Sequence-Level Knowledge Distillation*, EMNLP. Bastianelli et al. (2020), *SLURP*, EMNLP (CC BY 4.0). Biderman et al. (2023), Pythia. Hugging Face TRL distillation trainer docs; k3d, Kubernetes, SeaweedFS, Locust documentation.